MODUL PRAKTIKUM 08

ASYNCHRONOUS PROGRAMMING DENGAN KOTLIN COROUTINES & FLOW

suspend • viewModelScope • StateFlow • Flow Operators • Lifecycle-aware Collection

Pemrograman Aplikasi Perangkat Bergerak (CIF62041)
S1 Teknik Informatika · Fakultas Ilmu Komputer · Universitas Brawijaya



Durasi Praktikum: 170 menit | Tahun 2026

Tim Pengelola Mata Kuliah

1. Identitas dan Capaian Praktikum

Modul ini melanjutkan NavigationLab dari Modul Praktikum 07. Implementasi REST API yang sebelumnya menggunakan Retrofit callback direfactor menjadi suspend function, coroutine pada ViewModel, dan StateFlow sebagai state holder. Mahasiswa juga menggunakan Flow untuk membuat pencarian profil yang reaktif dan lifecycle-aware pada Jetpack Compose.

Sub-CPMK terkait

Mahasiswa mampu menerapkan komputasi asinkron berbasis Kotlin Coroutines dan Flow secara lifecycle-aware pada aplikasi Android. Fokus praktikum: suspend function, viewModelScope, StateFlow, operator Flow dasar, error handling, dan collectAsStateWithLifecycle().

Capaian Praktikum

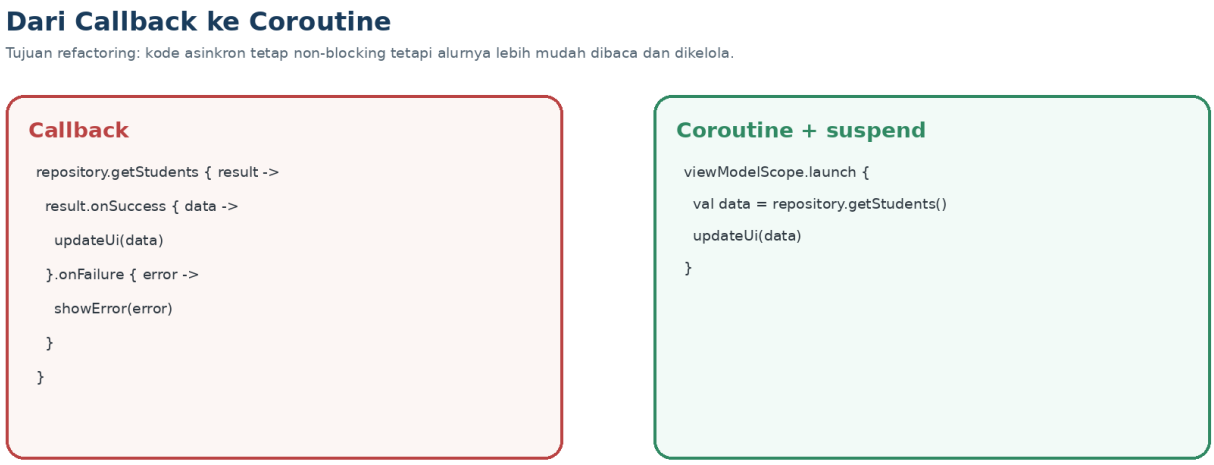
Setelah praktikum, mahasiswa mampu mengubah API callback menjadi suspend function, menjalankan pekerjaan asinkron melalui viewModelScope, mengekspos UI state menggunakan StateFlow, mengoleksi state secara lifecycle-aware pada Compose, serta memanfaatkan operator Flow untuk pencarian reaktif.

Indikator Keberhasilan

- StudentApiService menggunakan suspend function dan tidak lagi mengembalikan Call<T>.
- Remote data source dan repository mengekspos suspend function untuk operasi one-shot.
- ViewModel membuat coroutine menggunakan viewModelScope.launch.
- MutableStateFlow disimpan private dan diekspos sebagai StateFlow yang immutable.
- Composable mengoleksi StateFlow dengan collectAsStateWithLifecycle().
- Loading, success, dan error state tetap bekerja setelah refactoring.
- Pencarian memanfaatkan StateFlow query serta operator debounce, distinctUntilChanged, dan collectLatest.
- Navigation dan back stack dari modul sebelumnya tetap berfungsi.

3. Konsep Singkat: Coroutine, suspend, dan Flow

Coroutine adalah mekanisme concurrency pada Kotlin yang memungkinkan operasi asinkron ditulis dengan gaya kode yang lebih linear. Kata kunci suspend menandai fungsi yang dapat ditangguhkan tanpa memblokir thread. Pada arsitektur Android, operasi one-shot seperti request network umumnya diekspos sebagai suspend function, sedangkan data yang berubah dari waktu ke waktu dapat diekspos sebagai Flow.



Gambar 1. Perbandingan pola callback dan coroutine pada operasi asinkron.

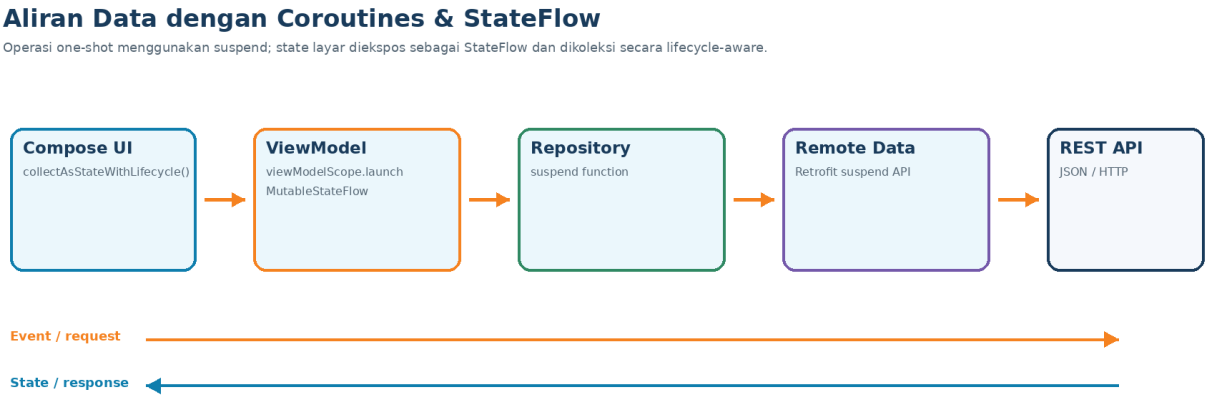
Istilah Inti

Bagian	Keterangan
suspend function	Fungsi yang dapat menangguhkan eksekusi coroutine tanpa memblokir thread pemanggil.
CoroutineScope	Batas lifecycle coroutine. Pada ViewModel digunakan viewModelScope.
viewModelScope	Scope bawaan ViewModel; coroutine dibatalkan ketika ViewModel dibersihkan.
Flow	Stream data asinkron yang dapat memancarkan lebih dari satu nilai.
StateFlow	Hot Flow yang selalu memiliki nilai state terbaru dan cocok untuk UI state.
MutableStateFlow	StateFlow yang dapat diubah oleh producer; sebaiknya tetap private di ViewModel.
collectLatest	Mengoleksi nilai terbaru dan membatalkan pekerjaan koleksi sebelumnya bila nilai baru datang.
collectAsStateWithLifecycle	Mengubah Flow/StateFlow menjadi Compose State dengan koleksi yang mengikuti lifecycle UI.

Prinsip Arsitektur

Data/business layer mengekspos suspend function untuk operasi one-shot dan Flow untuk data yang berubah. ViewModel membuat coroutine dan mengekspos state immutable ke UI.

4. Arsitektur Setelah Refactoring Coroutine



Gambar 2. Aliran event, suspend function, dan StateFlow pada NavigationLab.

Perubahan pada modul ini terutama terjadi pada data layer dan presentation layer. UI tidak mengetahui Retrofit atau coroutine detail. UI hanya mengirim event dan mengoleksi state. ViewModel mengatur kapan coroutine dijalankan dan kapan state diperbarui.

5. Langkah Kerja

Langkah 1 — Siapkan Project Modul 07

1. Buka NavigationLab hasil Modul Praktikum 07.
2. Jalankan aplikasi dan pastikan data REST API, Detail, Profile, About, error state, dan tombol Coba Lagi berfungsi.
3. Pastikan fitur pencarian berbasis tombol dari tugas Modul 07 sudah tersedia.
4. Buat checkpoint/commit sebelum refactoring.

Langkah 2 — Tambahkan Dependency Coroutine dan Lifecycle Compose

Gunakan versi yang kompatibel dengan project praktikum. Contoh berikut mempertahankan stack project sebelumnya (compileSdk/AGP dari modul awal) dengan Lifecycle 2.9.4, dan menggunakan kotlinx-coroutines 1.11.0.

app/build.gradle.kts

```
dependencies {
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.11.0")
    implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.9.4")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.9.4")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.9.4")
}
```

Catatan kompatibilitas

Jika project Anda sudah mendefinisikan versi Lifecycle melalui version catalog atau dependency Compose template, gunakan satu versi Lifecycle yang konsisten. Jangan menduplikasi versi berbeda dalam satu project.

Langkah 3 — Ubah Retrofit Service Menjadi suspend

Retrofit mendukung suspend function secara langsung. Karena itu Call<T> dan enqueue() tidak lagi diperlukan untuk endpoint yang digunakan pada modul ini.

data/remote/StudentApiService.kt

```
interface StudentApiService {

    @GET("users")
    suspend fun getUsers(
        @Query("limit") limit: Int = 10
    ): UserResponse

    @GET("users/{id}")
    suspend fun getUser(
        @Path("id") id: Int
    ): UserDto

    @GET("users/search")
    suspend fun searchUsers(
        @Query("q") query: String
    ): UserResponse
}
```

Hapus import retrofit2.Call. Retrofit akan menangguhkan coroutine sampai response selesai atau melempar exception ketika request gagal.

Langkah 4 — Refactor Remote Data Source

Remote data source menjadi lebih sederhana karena tidak lagi menerima callback. Setiap fungsi mengembalikan hasil langsung melalui suspend function.

data/remote/StudentRemoteDataSource.kt

```
class StudentRemoteDataSource(
    private val api: StudentApiService
) {
    suspend fun getStudents(): List<UserDto> =
        api.getUsers(limit = 10).users

    suspend fun getStudent(id: Int): UserDto =
        api.getUser(id)

    suspend fun searchStudents(query: String): List<UserDto> =
        api.searchUsers(query).users
}
```

Tidak perlu withContext(Dispatchers.IO) untuk Retrofit suspend
Retrofit menjalankan request HTTP secara asinkron. Pada modul ini kita tidak membungkus request Retrofit dengan withContext(Dispatchers.IO). Dispatcher IO tetap relevan untuk API blocking seperti operasi file atau library sinkron.

Langkah 5 — Refactor Repository Menjadi suspend Function

data/repository/StudentRepository.kt

```
interface StudentRepository {
    suspend fun getStudents(): List<Student>
    suspend fun getStudent(id: Int): Student
    suspend fun searchStudents(query: String): List<Student>
}
```

data/repository/DefaultStudentRepository.kt

```
class DefaultStudentRepository(
    private val remoteDataSource: StudentRemoteDataSource
) : StudentRepository {

    override suspend fun getStudents(): List<Student> =
        remoteDataSource.getStudents().map { it.toStudent() }

    override suspend fun getStudent(id: Int): Student =
        remoteDataSource.getStudent(id).toStudent()

    override suspend fun searchStudents(query: String): List<Student> =
        remoteDataSource.searchStudents(query).map { it.toStudent() }
}
```

Repository tetap menyembunyikan DTO dari presentation layer. ViewModel hanya bekerja dengan model Student.

Langkah 6 — Gunakan MutableStateFlow pada HomeViewModel

Ganti mutableStateOf dari modul sebelumnya menjadi MutableStateFlow. State yang dapat diubah hanya disimpan di dalam ViewModel, sedangkan UI menerima StateFlow read-only.

ui/home/HomeViewModel.kt — state dasar

```
class HomeViewModel(
    private val repository: StudentRepository
) : ViewModel() {

    private val _uiState = MutableStateFlow(HomeUiState())
    val uiState: StateFlow<HomeUiState> = _uiState.asStateFlow()

    init { loadStudents() }

    fun loadStudents() {
        viewModelScope.launch {
            _uiState.value = HomeUiState(isLoading = true)

            try {
                val students = repository.getStudents()
                _uiState.value = HomeUiState(students = students)
            } catch (e: CancellationException) {
                throw e
            } catch (e: Exception) {
                _uiState.value = HomeUiState(
                    errorMessage = e.message ?: "Gagal mengambil data"
                )
            }
        }
    }
}
```

Mengapa viewModelScope?
Coroutine yang memuat data layar dimulai oleh ViewModel. Ketika ViewModel dibersihkan, pekerjaan yang masih berjalan dibatalkan otomatis sehingga lifecycle pekerjaan mengikuti layar secara lebih aman.

Langkah 7 — Refactor DetailViewModel

ui/detail/DetailViewModel.kt — potongan utama

```
private val _uiState = MutableStateFlow(DetailUiState())
val uiState: StateFlow<DetailUiState> = _uiState.asStateFlow()

fun loadStudent(id: Int) {
    viewModelScope.launch {
        _uiState.value = DetailUiState(isLoading = true)

        try {
            val student = repository.getStudent(id)
            _uiState.value = DetailUiState(student = student)
        } catch (e: CancellationException) {
            throw e
        } catch (e: Exception) {
            _uiState.value = DetailUiState(
                errorMessage = e.message ?: "Data tidak tersedia"
            )
        }
    }
}
```

Langkah 8 — Koleksi State secara Lifecycle-aware di Compose

StateFlow jangan dibaca dengan .value langsung di UI untuk rendering reaktif. Gunakan collectAsStateWithLifecycle() pada composable yang memiliki ViewModel.

Potongan destination Home

```
val homeViewModel: HomeViewModel = viewModel(
    factory = HomeViewModelFactory(repository)
)
val homeUiState by homeViewModel.uiState.collectAsStateWithLifecycle()

HomeScreen(
    uiState = homeUiState,
    onStudentClick = { id ->
        navController.navigate(Routes.detail(id))
    },
    onRetry = homeViewModel::loadStudents
)
```

Lakukan pola yang sama pada DetailViewModel. Gunakan ViewModelProvider.Factory atau mekanisme dependency injection yang sudah dipakai pada project untuk membuat ViewModel; jangan membuat instance ViewModel secara manual dengan remember. collectAsStateWithLifecycle mulai mengoleksi saat lifecycle aktif dan berhenti saat UI tidak aktif, sehingga lebih aman untuk Compose.

Langkah 9 — Tambahkan Search Query sebagai StateFlow

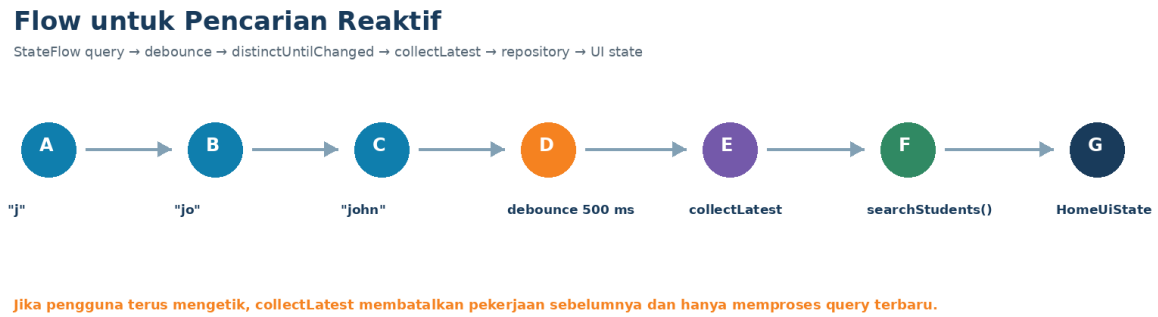
Tugas Modul 07 menggunakan tombol Cari. Pada modul ini query dibuat reaktif sehingga perubahan teks menjadi stream data.

Tambahan pada HomeViewModel

```
private val _searchQuery = MutableStateFlow("")
val searchQuery: StateFlow<String> = _searchQuery.asStateFlow()

fun onSearchQueryChange(query: String) {
    _searchQuery.value = query
}
```

Langkah 10 — Gunakan Operator Flow untuk Pencarian Reaktif



Gambar 3. Flow query pada pencarian reaktif.

Kita menggunakan debounce agar request tidak dikirim pada setiap karakter, distinctUntilChanged agar query sama tidak diproses ulang, dan collectLatest agar pekerjaan sebelumnya dibatalkan saat query baru datang.

HomeViewModel.kt — observeSearch()

```
@OptIn(FlowPreview::class)
private fun observeSearch() {
    viewModelScope.launch {
        searchQuery
            .debounce(500)
            .distinctUntilChanged()
            .collectLatest { query ->
                _uiState.value = _uiState.value.copy(
                    isLoading = true,
                    errorMessage = null
                )

                try {
                    val data = if (query.isBlank()) {
                        repository.getStudents()
                    } else {
                        repository.searchStudents(query)
                    }

                    _uiState.value = HomeUiState(students = data)
                } catch (e: CancellationException) {
                    throw e
                } catch (e: Exception) {
                    _uiState.value = HomeUiState(
                        errorMessage = e.message ?: "Pencarian gagal"
                    )
                }
            }
    }
}

init {
    observeSearch()
}
```

Setelah observeSearch() digunakan, hapus pemanggilan loadStudents() dari init agar request awal tidak dilakukan dua kali. Query awal kosong akan memuat daftar awal setelah debounce. Tambahkan import FlowPreview dan CancellationException dari kotlinx.coroutines; CancellationException harus dilempar kembali agar mekanisme pembatalan collectLatest tetap bekerja.

Langkah 11 — Hubungkan TextField ke Search Query

Potongan composable host Home

```
val uiState by homeViewModel.uiState.collectAsStateWithLifecycle()
val searchQuery by homeViewModel.searchQuery.collectAsStateWithLifecycle()

HomeScreen(
    uiState = uiState,
    searchQuery = searchQuery,
    onSearchQueryChange = homeViewModel::onSearchQueryChange,
    onStudentClick = { id -> navController.navigate(Routes.detail(id)) },
    onRetry = homeViewModel::loadStudents
)
```

Potongan HomeScreen.kt

```
OutlinedTextField(
    value = searchQuery,
    onValueChange = onSearchQueryChange,
    label = { Text("Cari profil") },
    modifier = Modifier.fillMaxWidth()
)
```

Tombol Cari dari Modul 07 dapat dihapus. Pencarian sekarang berjalan otomatis setelah pengguna berhenti mengetik sekitar 500 ms.

Langkah 12 — Perbaiki Retry agar Mengikuti Query Aktif

Jika error terjadi ketika pencarian aktif, retry sebaiknya memanggil request sesuai query yang sedang digunakan.

HomeViewModel.kt — retry()

```
fun retry() {
    val query = searchQuery.value

    viewModelScope.launch {
        _uiState.value = _uiState.value.copy(isLoading = true, errorMessage = null)

        try {
            val data = if (query.isBlank()) {
                repository.getStudents()
            } else {
                repository.searchStudents(query)
            }
            _uiState.value = HomeUiState(students = data)
        } catch (e: CancellationException) {
            throw e
        } catch (e: Exception) {
            _uiState.value = HomeUiState(
                errorMessage = e.message ?: "Request gagal"
            )
        }
    }
}
```

Pada HomeScreen ubah onRetry menjadi homeViewModel::retry.

Langkah 13 — Uji Cancellation secara Sederhana

1. Jalankan aplikasi dengan internet aktif.
2. Ketik j, lalu segera lanjutkan menjadi jo, joh, dan john sebelum 500 ms berlalu.
3. Perhatikan bahwa request tidak perlu dipanggil untuk setiap karakter; hasil akhir mengikuti query terbaru.
4. Ketik query lain dengan cepat untuk melihat collectLatest mengganti pekerjaan sebelumnya.
5. Kosongkan TextField dan pastikan daftar awal dimuat kembali.

Makna collectLatest
Ketika nilai baru dipancarkan, block collectLatest sebelumnya dibatalkan. Ini cocok untuk UI seperti pencarian yang hanya membutuhkan hasil terbaru.

6. Pengujian Praktikum

Tandai setiap kondisi berikut sebelum mengerjakan tugas akhir.

- ☐ Retrofit service tidak lagi mengembalikan Call<T>.
- ☐ Remote data source dan repository menggunakan suspend function.
- ☐ HomeViewModel dan DetailViewModel menggunakan viewModelScope.launch.
- ☐ MutableStateFlow bersifat private dan UI menerima StateFlow read-only.
- ☐ Home dan Detail mengoleksi state dengan collectAsStateWithLifecycle().
- ☐ Data awal dari REST API tetap tampil.
- ☐ Error state muncul ketika koneksi internet dimatikan.

- ☐ Retry bekerja tanpa crash.
- ☐ Pencarian berjalan otomatis setelah debounce.
- ☐ Query kosong memuat kembali daftar awal.
- ☐ Klik hasil pencarian tetap membuka Detail yang benar.
- ☐ Back stack Home → Detail → Back tetap benar.
- ☐ UI tidak memanggil Retrofit, repository, atau launch coroutine untuk business logic secara langsung.

Screenshot yang Wajib Diambil

- Screenshot 1 — StudentApiService berisi suspend function.
- Screenshot 2 — HomeViewModel dengan StateFlow dan viewModelScope.
- Screenshot 3 — Home menampilkan data awal API.
- Screenshot 4 — hasil pencarian reaktif setelah debounce.
- Screenshot 5 — Detail hasil REST API.
- Screenshot 6 — error state ketika internet dimatikan.
- Screenshot 7 — Logcat atau tampilan yang menunjukkan aplikasi tetap berjalan setelah retry.

7. Tugas Praktikum

Tambahkan fitur Refresh yang menggunakan coroutine dan mempertahankan pola StateFlow. Tujuan tugas adalah memastikan mahasiswa dapat memicu operasi asinkron dari event UI melalui ViewModel tanpa mengembalikan business logic ke composable.

Ketentuan Implementasi

1. Tambahkan tombol Refresh pada HomeScreen.

2. Event tombol harus memanggil fungsi refresh() di HomeViewModel.

3. refresh() harus menggunakan viewModelScope.launch.

4. Jika searchQuery kosong, refresh memuat daftar awal.

5. Jika searchQuery berisi teks, refresh mengulang pencarian dengan query aktif.

6. State isLoading dan errorMessage harus diperbarui dengan benar.

7. Jangan membuat CoroutineScope baru di HomeScreen.

8. Jangan menggunakan GlobalScope.

9. Setelah refresh selesai, aplikasi tetap dapat membuka Detail dan kembali ke Home.

10. Tambahkan satu screenshot sebelum dan sesudah Refresh pada laporan.

8. Luaran dan Format Laporan

A. Source Code

- Kumpulkan project Android Studio lengkap atau repository sesuai mekanisme kelas.
- Project harus memuat suspend Retrofit service, suspend repository, StateFlow pada ViewModel, lifecycle-aware collection, dan pencarian berbasis Flow.
- Hapus kode callback lama yang sudah tidak digunakan.
- Pastikan project dapat di-build.

B. Screenshot

Sertakan tujuh screenshot wajib dari bagian pengujian dan minimal dua screenshot fitur Refresh pada tugas praktikum.

C. Laporan Singkat

Laporan cukup 2–3 halaman di luar cover. Gunakan struktur berikut:

Bagian	Keterangan
1. Tujuan	Ringkas tujuan refactoring callback menjadi Coroutines dan Flow.
2. Refactoring	Jelaskan perubahan StudentApiService, remote data source, dan repository.
3. ViewModel	Jelaskan viewModelScope, MutableStateFlow, StateFlow, dan mekanisme UI state.
4. Lifecycle	Jelaskan alasan penggunaan collectAsStateWithLifecycle().
5. Flow Pencarian	Jelaskan debounce, distinctUntilChanged, dan collectLatest.
6. Hasil Pengujian	Masukkan screenshot loading, data, pencarian, error, retry, dan refresh.
7. Refleksi	Bandingkan callback Modul 07 dengan coroutine pada Modul 08: mana yang lebih mudah dibaca dan mengapa?

9. Troubleshooting Singkat

Gejala	Pemeriksaan
Unresolved reference collectAsStateWithLifecycle	Pastikan lifecycle-runtime-compose ditambahkan dan import androidx.lifecycle.compose.collectAsStateWithLifecycle digunakan.
Unresolved reference viewModelScope	Pastikan lifecycle-viewmodel-ktx/compose tersedia dan import androidx.lifecycle.viewModelScope digunakan.
Suspend function can be called only within coroutine	Panggil suspend function dari viewModelScope.launch atau suspend function lain.
Request terjadi dua kali saat aplikasi dibuka	Pastikan init tidak menjalankan loadStudents() sekaligus observeSearch() yang juga memuat query kosong.
Pencarian terlalu sering	Pastikan debounce berada sebelum collectLatest dan onValueChange hanya mengubah searchQuery.
UI tidak berubah	Pastikan _uiState.value diubah dan UI mengoleksi uiState dengan collectAsStateWithLifecycle.
Retry mengabaikan query	Gunakan searchQuery.value untuk menentukan request yang diulang.
CancellationException muncul saat request diganti	Jangan ubah cancellation menjadi error UI. Catch CancellationException secara terpisah lalu throw kembali agar structured cancellation tetap bekerja.
Data kosong setelah query	Periksa endpoint search, query, dan struktur UserResponse dari API.

10. Ringkasan

Pada praktikum ini NavigationLab berpindah dari pola callback menjadi Kotlin Coroutines. Retrofit API, remote data source, dan repository menggunakan suspend function untuk operasi one-shot. ViewModel

menjalankan pekerjaan melalui viewModelScope dan mengekspos UI state sebagai StateFlow. Compose mengoleksi state menggunakan collectAsStateWithLifecycle. Pencarian dikembangkan menjadi stream reaktif menggunakan StateFlow dan operator debounce, distinctUntilChanged, serta collectLatest.

Jembatan ke Modul Berikutnya
Setelah memahami data remote dan pemrograman asinkron, modul berikutnya dapat menambahkan penyimpanan lokal menggunakan DataStore dan Room Database. Coroutines dan Flow yang dipelajari di sini akan digunakan kembali saat mengakses database lokal.

11. Referensi

[1] Android Developers. Best practices for coroutines in Android. https://developer.android.com/kotlin/coroutines/coroutines-best-practices (diakses Oktober 2026).

[2] Android Developers. Use Kotlin coroutines with lifecycle-aware components. https://developer.android.com/topic/libraries/architecture/coroutines (diakses Oktober 2026).

[3] Android Developers. StateFlow and SharedFlow. https://developer.android.com/kotlin/flow/stateflow-and-sharedflow (diakses Oktober 2026).

[4] Android Developers. Lifecycle release notes. Lifecycle 2.9.4. https://developer.android.com/jetpack/androidx/releases/lifecycle (diakses Oktober 2026).

[5] Kotlin. kotlinx.coroutines releases. Version 1.11.0. https://github.com/Kotlin/kotlinx.coroutines/releases (diakses Oktober 2026).

[6] Square. Retrofit - A type-safe HTTP client for Android and Java. https://square.github.io/retrofit/ (diakses Oktober 2026).

[7] RPS Pemrograman Aplikasi Perangkat Bergerak (CIF62041), FILKOM Universitas Brawijaya, 2026.